

## JAVA 17 OCP · YDS STUDY SERIES

# Unit 12 · Modules — Technical Memory Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu not, [ana çift dilli dersin](#) Java Platform Module System (JPMS) kurallarını hızlı tekrar, karar tablosu ve komut analizi biçiminde sıkıştırır. Sorular özgün OCP tarzı çalışma sorularıdır; gerçek sınav sorusu değildir.

## 1. JPMS zihinsel modeli

Java 9 ile gelen JPMS, package'ları bir üst seviyede **module** içinde toplar. Descriptor'lı bir **explicit named module**'ın temel bileşenleri şunlardır:

```
module
├─ module-info.java      → module declaration
├─ bir veya daha fazla package
│   └─ public API
│   └─ internal implementation
```

Bir class'ın **public** olması tek başına başka module'dan erişim sağlamaz. Package'ın ayrıca **exports** edilmesi ve caller module'ın gerekli readability ilişkisine sahip olması gerekir.

**Memory tip:** “**public** class kapıdır; **exports** ise module duvarındaki geçittir.”

## 2. module-info.java

Basit bir descriptor:

```
module zoo.animal.feeding {
    exports zoo.animal.feeding;
}
```

- Dosya adı tam olarak **module-info.java** olmalıdır.
- Source root altında module'ın kökünde bulunur; bir package içine konmaz.
- Derlendiğinde **module-info.class** oluşur.
- Module declaration'da **class**, **interface** veya **package** keyword'ü kullanılmaz.
- Directive'ler herhangi bir sırada yazılabilir; fakat okunabilirlik için tutarlı bir sıra tercih edilir.

### 3. Directive karar tablosu

| Directive                           | Sorulan soru                                            | Etki                                                 |
|-------------------------------------|---------------------------------------------------------|------------------------------------------------------|
| <code>requires m;</code>            | Bu module hangi module'ı okumalı?                       | Current module, <code>m</code> yi okur               |
| <code>requires transitive m;</code> | Bu dependency consumer'lara da aktarılmalı mı?          | Current module'ı okuyanlar <code>m</code> yi de okur |
| <code>exports p;</code>             | Package compile-time ve normal runtime erişime açık mı? | Tüm okuyan module'lara public type erişimi           |
| <code>exports p to a,b;</code>      | Yalnız belirli module'lar erişsin mi?                   | Qualified export                                     |
| <code>opens p;</code>               | Deep reflection tüm module'lara açık mı?                | Runtime reflective access                            |
| <code>opens p to a,b;</code>        | Reflection yalnız belirli module'lara mı açık?          | Qualified open                                       |
| <code>uses s;</code>                | Bu module service arıyor mu?                            | Service consumer bildirimi                           |
| <code>provides s with i;</code>     | Bu module service implementation sağlıyor mu?           | Service provider bildirimi                           |

`java.base` dışındaki her named module, `java.base` module'ını implicitly okur. `String`, `Object`, `System` gibi temel type'lar için descriptor'a ayrıca bir dependency yazılmaz.

### 4. `requires` ve readability

```
module zoo.animal.care {
    requires zoo.animal.feeding;
}
```

Bu declaration, `zoo.animal.care` module'ının `zoo.animal.feeding` module'ını okumasını sağlar. Ancak erişilecek package, feeding module'ı tarafından export edilmemişse `requires` tek başına yeterli değildir.

```
caller requires target      [GEREKLİ] readability
target exports package     [GEREKLİ] accessibility
public class/member        [GEREKLİ] Java access control
```

Üç koşuldan biri eksikse erişim derlenmeyebilir.

### 5. `requires transitive`

```
module zoo.animal.care {
    requires transitive zoo.animal.feeding;
}

module zoo.staff {
    requires zoo.animal.care;
}
```

`zoo.staff`, `care` module'ını okuduğu için `feeding` module'ını da okuyabilir. Bu, `care` module'ının public API'sinin `feeding` type'larını dışarı taşıdığı durumlarda yararlıdır.

| Declaration                               | Direct consumer dependency'yi görür mü?                    |
|-------------------------------------------|------------------------------------------------------------|
| <code>requires feeding;</code>            | Hayır; consumer ayrıca <code>requires feeding</code> yazar |
| <code>requires transitive feeding;</code> | Evet; readability aktarılır                                |

**OCP trap:** `transitive` keyword'ü `exports` ile değil, yalnız `requires` ile kullanılır.

## 6. `exports` ve qualified export

Herkese export:

```
module zoo.animal.feeding {
    exports zoo.animal.feeding;
}
```

Yalnız seçili module'lara export:

```
module zoo.animal.feeding {
    exports zoo.animal.feeding.internal to zoo.staff, zoo.vet;
}
```

Qualified export dışındaki named module'lar package'ın public type'larını normal yolla kullanamaz. Export, package'ın `private` veya package-private member'larını public hale getirmez.

**Memory tip:** “`exports ... to` compile-time dost listesidir.”

## 7. `opens`, qualified open ve `open module`

```
module zoo.animal.care {
    opens zoo.animal.care.details;
    opens zoo.animal.care.secret to zoo.staff;
}
```

`opens`, normal source-code erişimi sağlamaz; deep reflection için runtime erişimi sağlar. Framework'lerin private field veya constructor'lara reflection ile ulaşması buna örnektir.

```
open module zoo.data {
    exports zoo.data.api;
}
```

`open module`, module içindeki bütün package'ları deep reflection'a açar. Normal erişim için yine `exports` gerekir.

Bir `open module` declaration'ı ayrıca `opens` directive'i içeremez. Bütün package'lar zaten açıldığı için bu kullanım **Does not compile**.

| Amaç                                 | Directive                |
|--------------------------------------|--------------------------|
| Public API'yi normal kodla kullanmak | <code>exports</code>     |
| Private member'lara deep reflection  | <code>opens</code>       |
| Her package'ı reflection'a açmak     | <code>open module</code> |

**OCP trap:** `opens` “package'ı import edilebilir yapar” demek değildir.

## 8. İlk module'ı derlemek

Örnek source tree:

```
src/
├── zoo.animal.feeding/
│   ├── module-info.java
│   └── zoo/animal/feeding/Task.java
```

Tek module:

```
javac -d mods/zoo.animal.feeding \
  src/zoo.animal.feeding/module-info.java \
  src/zoo.animal.feeding/zoo/animal/feeding/Task.java
```

Birden fazla module source tree:

```
javac --module-source-path src \
  -d mods \
  -m zoo.animal.feeding,zoo.animal.care
```

| Option                            | Short form      | Görev                            |
|-----------------------------------|-----------------|----------------------------------|
| <code>--module-path</code>        | <code>-p</code> | Dependency module'larının konumu |
| <code>--module-source-path</code> | Yok             | Module source root pattern'i     |
| <code>--module</code>             | <code>-m</code> | Derlenecek module listesi        |
| <code>-d</code>                   | <code>-d</code> | Class output directory           |

`javac -d <directory>` compiled class output'unu seçer. Buna karşılık `java -d <module>` seçilen module'ı açıklar; `java --describe-module` ile aynı işlevdedir. Aynı kısa option'ı tool adından bağımsız yorumlama.

## 9. Modular program çalıştırmak

```
java --module-path mods \
  --module zoo.animal.talks/zoo.animal.talks.Peacocks
```

Kısa biçim:

```
java -p mods -m zoo.animal.talks/zoo.animal.talks.Peacocks
```

`module/class` ayırıcısı slash (/), fully qualified class name içindeki package ayırıcısı dot (.) karakteridir.

```
zoo.animal.talks/zoo.animal.talks.Peacocks
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
module           package + class
```

**OCP trap:** `java -m module/package/Class` yanlış; class name slash ile değil dot ile yazılır.

## 10. Modular JAR oluşturmak

```
jar --create \
  --file zoo.animal.feeding.jar \
  -C mods/zoo.animal.feeding .
```

Kısa legacy biçim:

```
jar -cvf zoo.animal.feeding.jar -C mods/zoo.animal.feeding .
```

Modular JAR, root'ta `module-info.class` içerir. Main class manifest/module metadata'sına kaydedilebilir:

```
jar --create \
  --file zoo.animal.talks.jar \
  --main-class zoo.animal.talks.Peacocks \
  -C mods/zoo.animal.talks .
```

Bu durumda:

```
java -p zoo.animal.talks.jar -m zoo.animal.talks
```

Main class belirtilmemişse `-m module/package.Main` gerekir.

## 11. Module keşfetme ve açıklama

Built-in module listesi:

```
java --list-modules
```

Bir module'ı açıklamak:

```
java --describe-module java.sql
java -p mods --describe-module zoo.animal.care
```

JAR descriptor'ını açıklamak:

```
jar --describe-module --file zoo.animal.care.jar
```

Output'ta şu kavramlar görülebilir:

- `requires`: dependency/readability
- `exports`: dışarı açılan package
- `contains`: export edilmeyen içerilen package
- `uses / provides`: service ilişkileri
- `requires java.base mandated`: implicit temel dependency

Exact output sırası veya JDK patch version numarası ezberlenmemelidir.

## 12. Service Provider Interface

API module:

```
module zoo.tours.api {  
    exports zoo.tours.api;  
}
```

```
package zoo.tours.api;  
  
public interface Tour {  
    String name();  
}
```

Service locator:

```
module zoo.tours.reservations {  
    requires zoo.tours.api;  
    exports zoo.tours.reservations;  
    uses zoo.tours.api.Tour;  
}
```

Provider:

```
module zoo.tours.agency {  
    requires zoo.tours.api;  
    provides zoo.tours.api.Tour  
        with zoo.tours.agency.TourImpl;  
}
```

Provider implementation package'ının export edilmesi gerekmez. Locator, implementation'ı doğrudan instantiate etmek yerine service contract üzerinden bulur; consumer locator'ın exported API'sini çağırır.

## 13. ServiceLoader

```
ServiceLoader<Tour> loader = ServiceLoader.load(Tour.class);

for (Tour tour : loader) {
    System.out.println(tour.name());
}
```

Provider metadata'sına önce bakmak:

```
List<Tour> tours = loader.stream()
    .map(ServiceLoader.Provider::get)
    .toList();
```

Service graph'ında:

- Service locator, API module'ını **requires** eder, service'i **uses** eder ve consumer'ın çağıracağı locator package'ını gerekiyorsa **exports** eder.
- Consumer, source code'da doğrudan kullandığı API/locator module'larını **requires** eder. **ServiceLoader** çağırısını locator'a bırakıyorsa kendisi **uses** yazmaz.
- Provider, API module'ını **requires** eder ve implementation'ı **provides ... with ...** ile bildirir.
- Locator veya consumer concrete provider module'ını **requires** etmez.
- Yeni provider module path'e eklendiğinde locator onu keşfedebilir.

**OCP trap:** **provides Implementation with Interface** sırası yanlıştır. Doğrusu **provides Interface with Implementation**dir.

## 14. Explicit named, automatic ve unnamed module

OCP karşılaştırmalarında “named module” çoğunlukla descriptor'lı explicit named module anlamında kullanılır. Java API terminolojisinde automatic module de bir ada sahip olduğu için named module'dür; aşağıdaki tabloda iki tür ayrı gösterilir.

| Özellik                         | Explicit named                                | Automatic                                          | Unnamed                                     |
|---------------------------------|-----------------------------------------------|----------------------------------------------------|---------------------------------------------|
| Konum                           | Module path                                   | Module path                                        | Classpath                                   |
| Effective <b>module-info</b>    | Var                                           | Yok                                                | Varsa yok sayılır                           |
| İsim                            | Descriptor'da                                 | Manifest/filename'dan                              | İsim yok                                    |
| Package erişimi                 | Yalnız <b>exports/opens</b> ile bildirilenler | Tüm package'lar export ve open kabul edilir        | Tüm package'lar export ve open kabul edilir |
| Explicit dependency declaration | Descriptor'da <b>requires</b>                 | Yok                                                | Yok                                         |
| Named module'ları okuma         | Declared graph                                | Configuration'daki bütün resolved named module'lar | Bütün resolved named module'lar             |
| Unnamed module'ı okuma          | Default olarak hayır                          | Evet                                               | Classpath dünyasını okur                    |

Explicit named module güçlü encapsulation sağlar. Automatic ve unnamed module'lar, legacy migration'ı aşamalı yapabilmek için geçiş mekanizmalarıdır. `--add-reads` gibi command-line istisnaları verilmedikçe explicit named module, unnamed module'ı okuyamaz. Automatic module ise migration uyumluluğu için runtime'da JVM'deki her unnamed module'ı da okur. Unnamed module'ın package'ları açık olsa bile explicit named caller'da readability yoksa normal erişim yine sağlanmaz.

## 15. Automatic module adı

Tercih sırası:

- 1 Manifest'teki `Automatic-Module-Name`
- 2 JAR filename'ından türetilen ad

Filename türetim algoritması:

- 1 `.jar` uzantısı kaldırılır.
- 2 Filename'da bir `-` işaretini bir veya daha fazla rakam ve ardından dot ya da filename sonunun izlediği ilk version eşleşmesi varsa module adı, bu eşleşmeyi başlatan `-` karakterinden önceki bölümden türetilir.
- 3 Harf/rakam dışındaki her karakter dot'a dönüştürülür.
- 4 Tekrarlanan dot'lar tek dot yapılır; baştaki ve sondaki dot'lar temizlenir.

Örneğin `acme-utils-2.1.jar`, manifest adı yoksa `acme.utils` automatic module adını alır.

Filename değişikçe derived name değişebileceğinden, library producer için stable

`Automatic-Module-Name` daha güvenlidir.

**OCP trap:** Automatic module'da `module-info.java` bulunmaz. Descriptor eklenirse artık explicit named module olur.

## 16. Unnamed module ve classpath

Classpath üzerindeki code unnamed module içinde değerlendirilir:

```
java -cp libs/*:classes com.example.Main
```

Bir modular JAR classpath'e konursa descriptor'ı module resolution için kullanılmaz; JAR normal classpath code'u gibi unnamed module'ın parçası olur.

Explicit named module, descriptor içinde unnamed module'ı adla `requires` edemez; çünkü unnamed module'ın adı yoktur ve default readability ilişkisi de bulunmaz. Automatic module bu migration engelini azaltmak için runtime'da unnamed module'ı okuyabilir. Buna karşılık legacy classpath application'ı, uygun biçimde resolved/export edilmiş named module API'lerine erişebilir.

## 17. Split package ve cyclic dependency

Standart tek-layer application düzeninde aynı package'ı birden fazla resolved named module'a bölmek **split package** problemidir; resolution/layer tanımlama başarısız olabilir. OCP sorularında bu yapı geçersiz kabul edilir.

Module cycle da geçersizdir:

```
module zoo.cats {
    requires zoo.dogs;
}

module zoo.dogs {
    requires zoo.cats;
}
```

**Sonuç: Does not compile.** Module graph'ta cycle oluşur. Çözüm; ortak API'yi üçüncü module'a taşımak, dependency yönünü değiştirmek veya module sınırlarını yeniden tasarlamaktır.

## 18. Migration stratejileri

### Bottom-up migration

- 1 En alt dependency'den başlanır.
- 2 O JAR explicit named module'a çevrilir.
- 3 Onu kullanan üst katmana geçilir.
- 4 Application'a kadar tekrarlanır.

Ara durumda migrate edilmiş alt katmanlar module path'te explicit named module'dür; henüz migrate edilmemiş üst katmanlar, application dahil, classpath'te unnamed module olarak kalabilir. Alt named module'ların application graph'ında resolved olması gerekir; launcher bağlamında bunun için gerektiğinde `--add-modules` kullanılır.

### Top-down migration

- 1 Descriptor'sız legacy JAR'lar module path'e alınarak automatic module olur; zaten modular olan JAR'lar explicit named kalır.
- 2 En üst application explicit named module'a dönüştürülür.
- 3 Dependency'ler yukarıdan aşağıya sırayla explicit named module yapılır.

Ara durumda explicit named application, henüz descriptor eklenmemiş dependency'leri stable/derived automatic module adlarıyla `requires` eder.

| Durum                                                                 | Daha doğal yaklaşım |
|-----------------------------------------------------------------------|---------------------|
| Alt library'leri kontrol ediyorsun                                    | Bottom-up           |
| Application'ı kontrol ediyor, dependency'leri hemen değiştiremiyorsun | Top-down            |

Migration sırasında module adı değişiklikleri, split package'lar ve illegal JDK internal API kullanımları ayrıca incelenmelidir.

## 19. `jdeps`

Özet dependency analizi:

```
jdeps -s zoo.animal.care.jar
```

Module path ile:

```
jdeps --module-path mods -s zoo.animal.care.jar
```

JDK internal API kontrolü:

```
jdeps --jdk-internals legacy.jar
```

`jdeps` static analysis yapar. Reflection ile String'den yüklenen class'ları, runtime plugin'lerini veya çalışmayan code path'lerin bütün davranışını eksiksiz garanti etmez.

**OCP trap:** `jdeps --module-path` option'ının `-p` kısa biçimi yoktur. `jdeps -p`, package filter anlamındaki farklı bir option'dır.

**Memory tip:** “`jdeps` dependency'yi keşfeder; descriptor'ı veya tasarımı sen doğrularsın.”

## 20. `jmod`

Temel işlemler:

```
jmod create --class-path mods/zoo.animal.feeding \
  zoo.animal.feeding.jmod

jmod describe zoo.animal.feeding.jmod
jmod list zoo.animal.feeding.jmod
```

JMOD; class dosyalarına ek olarak native library, command ve configuration gibi içerikler taşıyabilir. Normal application dağıtımında JAR daha yaygındır. `java` launcher bir `.jmod` dosyasını doğrudan application module'ı olarak çalıştırmaz; JMOD özellikle JDK image üretim zincirinde kullanılır.

## 21. `jlink`

Yalnız gerekli module'ları içeren custom runtime:

```
jlink \
  --module-path "$JAVA_HOME/jmods:mods" \
  --add-modules zoo.animal.talks \
  --output zoo-runtime
```

Generated launcher:

```
zoo-runtime/bin/java \
  -m zoo.animal.talks/zoo.animal.talks.Peacocks
```

Önemli noktalar:

- Root module'ların ordinary `requires` ve `requires transitive` kenarlarıyla oluşan resolved dependency closure'ı image'a dahil edilir.
- Output directory önceden mevcut olmamalıdır.
- Custom image platform-specific'tir.
- `jlink`, explicit named module graph'ı ile çalışır; classpath tabanlı unnamed application doğrudan root module değildir.
- Automatic module'lar custom runtime image'a bağlanamaz; `jlink` için dependency graph'ının explicit module'lara dönüştürülmesi gerekir.

## 22. Command-line hızlı karşılaştırma

| Tool               | Ana görev                        | Sık option                                               |
|--------------------|----------------------------------|----------------------------------------------------------|
| <code>javac</code> | Source compile                   | <code>-d, -p, --module-source-path, -m</code>            |
| <code>java</code>  | Program çalıştır / module keşfet | <code>-p, -m, --list-modules, --describe-module</code>   |
| <code>jar</code>   | JAR oluştur / açıkla             | <code>--create, --file, -d / --describe-module</code>    |
| <code>jdeps</code> | Static dependency analizi        | <code>-s, --module-path, --jdk-internals</code>          |
| <code>jmod</code>  | JMOD oluştur / listele           | <code>create, describe, list</code>                      |
| <code>jlink</code> | Custom runtime image             | <code>-p / --module-path, --add-modules, --output</code> |

## 23. Compile-time, runtime ve output ayrımı

| Durum                                                                        | Sınıflandırma                                           |
|------------------------------------------------------------------------------|---------------------------------------------------------|
| <code>javac</code> sırasında <code>requires</code> edilen module bulunamıyor | <b>Does not compile</b>                                 |
| Precompiled application launch edilirken required module bulunamıyor         | Resolution/launcher error; <code>main()</code> başlamaz |
| Source code doğrudan erişiyor fakat target package caller'a export edilmemiş | <b>Does not compile</b>                                 |
| Module graph cyclic                                                          | <b>Does not compile</b>                                 |
| Yanlış <code>module/class</code> launcher syntax'ı                           | Launcher error; program başlamaz                        |
| Provider yok, <code>ServiceLoader</code> iteration empty                     | Derlenir; boş sonuç mümkündür                           |
| <code>jlink</code> output directory zaten var                                | Tool runtime error                                      |
| Valid module main class <code>println("ok")</code>                           | Derlenir ve <code>ok</code> yazdırır                    |

OCP sorusunda önce hangi tool'un çalıştığını belirle. Compiler error ile launcher/tool error aynı kategori değildir.

## 24. OCP çözüm algoritması

- 1 Code classpath'te mi module path'te mi?
- 2 Descriptor var mı? Module type'ını belirle.
- 3 Module/package/class adlarını ayrı işaretle.
- 4 Caller'ın readability ilişkisi var mı?
- 5 Target package `exports` edilmiş mi; qualified ise caller listede mi?
- 6 Soru reflection ise `opens` koşulunu ayrıca kontrol et.
- 7 Service ise `uses` ve `provides ... with ...` yönlerini çiz.
- 8 Graph'ta missing module, duplicate dependency, split package veya cycle ara.
- 9 Command'da tool-specific option ve path separator'larını kontrol et.
- 10 Sonucu **Does not compile**, tool/launcher error veya başarılı output diye sınıflandır.

## 25. Mini quiz · Özgün OCP tarzı çalışma soruları

### Soru 1

`zoo.staff` module'ının non-empty `zoo.staff` package'ını içerdiğini varsayın.

```
module zoo.staff {  
    exports zoo.staff to zoo.visitor;  
}
```

Hangisi doğrudur?

- A. `zoo.staff` package'ı tüm module'lara açılır.
- B. Readability ilişkisi de varsa `zoo.visitor`, public type'lara normal erişim sağlayabilir.
- C. `zoo.visitor`, private field'lara reflection ile kesin erişir.
- D. Declaration derlenmez.

### Soru 2

```
module care {  
    requires transitive feeding;  
}  
module staff {  
    requires care;  
}
```

`staff` module'ı `feeding`'in exported API'sini okuyabilir mi?

### Soru 3

Aşağıdaki command'ın problemi nedir?

```
java -p mods -m zoo.talks/zoo/animal/talks/Show
```

#### Soru 4

Bir JAR module path'te, `module-info.class` içermiyor ve manifest'te `Automatic-Module-Name: com.example.lib` içeriyor. Module type'ı ve adı nedir?

#### Soru 5

Bir modular JAR classpath'e konursa hangi module type'ı gibi davranır?

#### Soru 6

```
module locator {  
    requires tours.api;  
    provides tours.api.Tour with locator.TourImpl;  
}
```

Bu module'ın yalnız `ServiceLoader` ile service arayan locator olduğu belirtiliyor. Descriptor bu rol için doğru mudur? Neden?

#### Soru 7

Provider module'ının implementation package'ını export etmesi zorunlu mudur?

#### Soru 8

```
module alpha { requires beta; }  
module beta { requires alpha; }
```

Compile-time, runtime veya output sonucu nedir?

#### Soru 9

`exports p;` ile `opens p;` arasındaki temel farkı açıklayın.

#### Soru 10

Hangi tool illegal JDK internal API kullanımını aramak için uygundur?

- A. `jar --describe-module`
- B. `jdeps --jdk-internals`
- C. `jlink --add-modules`
- D. `java --list-modules`

#### Soru 11

`jlink` command'ında mevcut ve dolu bir output directory verilirse bunun compile-time error olduğu söylenebilir mi?

#### Soru 12

Bottom-up migration hangi dependency seviyesinden başlar?

**Soru 13**

`requires transitive` ile ilgili iki doğru seçeneği seçin.

- A. Yalnız `open module` içinde kullanılabilir.
- B. Readability'yi current module'ın consumer'larına aktarır.
- C. Package'ı reflection'a açar.
- D. `requires` etkisini de içerir.

**Soru 14**

```
module secret.data {  
    opens secret.data.model to json.framework;  
}
```

`other.app, secret.data.model.Record` type'ını source code'da doğrudan import edebilir mi?

**Soru 15**

Automatic module ve unnamed module arasındaki iki temel konum/isim farkını yazın.

## Cevaplar ve açıklamalar

- 1 **B.** `exports ... to` qualified export'tur. Normal public API erişimini yalnız listedeki module'a verir. Reflection için `opens` gerekir.
- 2 **Evet.** `care`, feeding'i transitive require ettiği için `care`'i okuyan staff feeding readability'sini de kazanır. Package'ın feeding tarafından export edilmiş olması ayrıca gerekir.
- 3 Main class package bölümü slash ile yazılmıştır. Doğrusu `zoo.talks/zoo.animal.talks.Show` biçimidir. Command mevcut haliyle programı başlatmaz.
- 4 **Automatic module**, adı `com.example.lib`. Manifest adı filename türetimine göre önceliklidir.
- 5 **Unnamed module** davranışı gösterir. Classpath, modular descriptor'ı module resolution için kullanmaz.
- 6 Hayır. Locator service'i `uses tours.api.Tour;` ile bildirir. `provides ... with ...` provider module'ın görevidir. Service aramayı locator'a bırakan ayrı consumer'ın `uses` yazması da gerekmez.
- 7 Hayır. JPMS service mechanism implementation'ı descriptor üzerinden bulabilir. Provider genellikle yalnız API module'ını require eder ve implementation'ı `provides` ile bildirir.
- 8 **Does not compile.** Named module graph'ında cyclic dependency vardır.
- 9 `exports`, public type'ların normal compile-time/runtime erişimini sağlar. `opens`, deep reflection için runtime erişimi sağlar; normal import hakkı vermez.
- 10 **B.** `jdeps --jdk-internals` static analysis ile internal JDK API referanslarını raporlar.
- 11 Hayır. Source compilation yapılmıyorsa compile-time error değildir; `jlink` tool execution sırasında hata verir.
- 12 En alt/leaf dependency'den başlar ve application'a doğru yukarı ilerler.
- 13 **B ve D.** Transitive requires, normal `requires` readability'sini içerir ve bunu current module'ı okuyanlara aktarır.
- 14 Hayır. Qualified `opens`, yalnız `json.framework` için reflection erişimi verir. `exports` bulunmadığı için normal import sağlamaz.
- 15 Automatic module module path'tedir ve manifest/filename'dan bir adı vardır. Unnamed module classpath'tedir ve declaration'da kullanılabilecek bir adı yoktur.

## Son tekrar kartı

```
NORMAL API      exports
REFLECTION      opens
DEPENDENCY      requires
PROPAGATION     requires transitive
CONSUMER        uses
PROVIDER        provides Service with Impl

module path + descriptor      explicit named
module path - descriptor     automatic
classpath                     unnamed

explicit named  -X-> unnamed  (default)
automatic      ---> unnamed  (runtime compatibility)
unnamed        ---> resolved named modules

javac  compile
java   run/discover
jar    package/describe
jdeps  analyze dependencies
jmod   package for image/native content
jlink  custom runtime
```